

High Performance Computing – Lab 3

Orazgali Azamat, BDA-2402

Part 1: Monte Carlo Approximation of Pi

It generates N random points in the square $-1 \leq x \leq 1$ using `std::mt19937` seeded with 42, splits them into 4 equal groups (for 4 processors), counts how many points fall inside the unit circle in each group, sums the local counts into M, and approximates pi as $4 * M / N$. For N = 1,000,000 the program printed:

N = 1000000

Total inside (M) = 785607

Pi approximation = 3.14243

Part 2: Effect of Problem Size

N	Approximation of Pi
10^3	3.152
10^4	3.1708
10^5	3.14668
10^6	3.14243

1. Does the approximation generally improve when N increases?

Yes. As N grows the approximation gets closer to the true value of pi (3.14159...), even though it does not improve perfectly smoothly at every step because the points are random.

2. Does the program have to store all generated points?

No. Only a running count of points and points inside the circle is needed

3. Can one generated point be processed without knowing the result for another point?

Yes. Each point is checked independently with $x * x + y * y \leq 1$, so no point depends on the result of another.

Part 3: Partitioning

N = 1,000,000, divided equally among 4 processors:

Processor	Number of Points	Local Result
P0	250,000	M0
P1	250,000	M1

P2	250,000	M2
P3	250,000	M3

1. How many points should each processor generate?

250,000 points each (1,000,000 / 4).

2. Can the processors perform their work independently?

Yes, since each point is generated and tested independently of all other points.

3. Domain decomposition or functional decomposition?

Domain decomposition, because the data (the set of generated points) is split among processors that all perform the same operation.

4. Does one processor need the coordinates generated by another processor?

No, each processor only needs its own generated coordinates.

Part 4: Simulating Multiple Processors

For N = 1,000,000 the program printed the following local results:

Processor 0: points = 250000, inside = 196427

Processor 1: points = 250000, inside = 196282

Processor 2: points = 250000, inside = 196271

Processor 3: points = 250000, inside = 196627

Part 5: Reduction

$M = M0 + M1 + M2 + M3 = 196427 + 196282 + 196271 + 196627 = 785607$

Pi approximation = $4 * 785607 / 1000000 = 3.14243$

1. Is calculating each Mp an independent computation?

Yes, each processor calculates its own Mp using only its own points

2. Is calculating the final M an independent computation?

No, it needs the results (Mp values) from every processor, so it is not independent

3. What information must processors communicate at the end?

Each processor must send its local count of points inside the circle (Mp)

4. Local or global communication?

Global communication, since the final sum needs data from all processors

5. What is this operation called in parallel computing?

A reduction (sum reduction).

Part 6: Agglomeration

1. How many tasks if each task processes 10,000 points?

$1,000,000 / 10,000 = 100$ tasks.

2. One task per point: fine-grained or coarse-grained?

Fine-grained parallelism.

3. Why can combining many small tasks into larger tasks reduce overhead?

Creating, scheduling and synchronizing a task costs time on its own. If every task only does one point, this overhead can be larger than the actual work. Combining points into bigger tasks means the fixed overhead is paid far fewer times.

4. What disadvantage could occur if the tasks become too large?

With too few, too large tasks, there may not be enough tasks to keep all processors busy, and the workload could become unbalanced if the tasks are not exactly equal in size.

Part 7: Mapping

Eight equal tasks T0-T7 assigned to four processors:

Processor	Assigned Tasks
P0	T0, T1
P1	T2, T3
P2	T4, T5
P3	T6, T7

1. How many tasks does each processor receive?

Two tasks each.

2. Is the workload balanced?

Yes, since the eight tasks are equal in size and each processor gets exactly two of them.

3. What happens to the total execution time if one processor receives much more work than the others?

The total execution time increases, because all processors must wait for the slowest (most loaded) one to finish. The extra processors sit idle after finishing their smaller share, so the achievable speedup drops.

Part 8: Parallel Program Design

1. Partitioning:

The set of N random points (or, equivalently, the loop over i) should be divided into independent tasks, each responsible for generating and checking a subset of the points.

2. Communication:

Each task only needs to communicate its local count of points inside the circle (M_p) at the end, to be combined into the total M .

3. Agglomeration:

Many individual point-generation tasks can be combined into larger tasks because each point check is extremely small work, and grouping many points per task reduces task-management overhead relative to the actual computation and current setup power

4. Mapping:

The combined tasks should be split as evenly as possible among the available processors, giving each processor roughly the same number of points/tasks so the workload stays balanced.

Part 9: Short Analysis**1. Time complexity of the loop in Big-O notation:**

$O(N)$.

2. If N is doubled, approximately how much additional work must be performed?

Approximately twice as much work, since the loop runs proportionally to N .

3. Which part of the algorithm can be executed independently by different processors?

Generating a point and checking whether it lies inside the circle (the body of the loop) can be done independently for each point.

4. Which operation requires the results from all processors?

The final reduction that sums the local inside-counts into the total M .